PYTHON PARA VIDEOJUEGOS CON PYGAME · LECCIÓN 4

Enemigos que caen - clases y grupos

Objetivo: Al terminar puedes escribir una clase de pygame.sprite.Sprite, crear muchos enemigos con ella y manejarlos juntos con un Group.
Antes de empezar:

- Lección 3 - el ovni que dispara.
- Saber qué es una función (def).

Para copiar el código: al copiar desde un PDF, muchos visores pierden los espacios del principio de cada línea, y en Python esos espacios importan. En la misma carpeta de esta guía está cada paso como archivo `.py`: ábrelo y copia de ahí.

Paso a paso

Enemigos que caen: clases y grupos

Llegó la hora de los enemigos, y van a ser muchos. Para no repetir código, vas a escribir una clase: un molde para fabricar enemigos. Cada enemigo va a tener su propia posición y su propia velocidad. Y un grupo de pygame los va a mover y dibujar a todos juntos.

Una clase es un molde; __init__ prepara cada objeto y self es ese objeto

Primero, junto a las otras imágenes, carga la del enemigo, igual que la del ovni, con convert_alpha para respetar su transparencia. La cargas una sola vez, aquí arriba, y todos los enemigos van a compartirla. Ahora escribe una clase: un molde que junta los datos de un enemigo y lo que sabe hacer. Se llama Enemy, con mayúscula, como se acostumbra con las clases. Entre paréntesis va pygame.sprite.Sprite: tu clase hereda, o sea, recibe gratis lo que ya sabe hacer un sprite. __init__ es una función especial, que corre sola cada vez que creas un enemigo. self es ese enemigo concreto: lo que guardas en self es solo suyo. La línea super().__init__() prepara la parte heredada, y va siempre primero. Después le das su imagen y su Rect, en self.image y self.rect: esos nombres exactos son los que el grupo busca para dibujarlo.

Tu `game.py` queda así — las líneas resaltadas son las nuevas o cambiadas (archivo `paso-1.py`):

```
import pygame

pygame.init()
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("Most Wanted UFO Mini")
clock = pygame.time.Clock()

background = pygame.image.load("assets/background_4.png").convert()
player_image = pygame.image.load("assets/player_1.png").convert_alpha()
bullet_image = pygame.image.load("assets/Bullet.gif").convert_alpha()
enemy_image = pygame.image.load("assets/enemy_1.png").convert_alpha()
shoot_sound = pygame.mixer.Sound("assets/shoot.wav")
```

```
class Enemy(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = enemy_image
        self.rect = self.image.get_rect()


player = player_image.get_rect(midbottom=(400, 590))
speed = 6
bullets = []
cooldown = 0

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT]:
        player.x -= speed
    if keys[pygame.K_RIGHT]:
        player.x += speed
    player.clamp_ip(screen.get_rect())

    if cooldown > 0:
        cooldown -= 1
    if keys[pygame.K_SPACE] and cooldown == 0:
        bullets.append(bullet_image.get_rect(midbottom=player.midtop))
        shoot_sound.play()
        cooldown = 15

    for bullet in bullets:
        bullet.y -= 10
    bullets = [b for b in bullets if b.bottom > 0]

    screen.blit(background, (0, 0))
    for bullet in bullets:
        screen.blit(bullet_image, bullet)
    screen.blit(player_image, player)
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

Los métodos reset y update dan a cada enemigo su posición y su movimiento

Arriba de todo, importa random, el módulo de Python para elegir números al azar. Por costumbre, las importaciones van al principio del archivo, antes de cualquier otra cosa. Al final de __init__, agrega self.reset(). Un método es una función dentro de la clase; reset lo escribes justo abajo, y lo llamas aquí para que cada enemigo nuevo se coloque en su lugar apenas nace. reset recibe self, como todo método. Pone la base del enemigo en una equis al azar entre veinte y setecientos ochenta. Su y también es al azar, entre menos quinientos y menos cuarenta: negativa, o sea, por encima de la pantalla. Así no llegan todos a la vez. Además le da una velocidad de uno o dos. Como cada uno guarda su propia velocidad en self, no van a caer todos igual. Y update baja al enemigo tantos

píxeles como diga su velocidad. Si su borde de arriba pasa de seiscientos, ya salió por abajo: llama otra vez a reset y vuelve a empezar arriba. Todavía nadie llama a update; eso lo va a hacer el grupo, en cada cuadro.

Tu `game.py` queda así — las líneas resaltadas son las nuevas o cambiadas (archivo `paso-2.py`):

```
import random

import pygame

pygame.init()
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("Most Wanted UFO Mini")
clock = pygame.time.Clock()

background = pygame.image.load("assets/background_4.png").convert()
player_image = pygame.image.load("assets/player_1.png").convert_alpha()
bullet_image = pygame.image.load("assets/Bullet.gif").convert_alpha()
enemy_image = pygame.image.load("assets/enemy_1.png").convert_alpha()
shoot_sound = pygame.mixer.Sound("assets/shoot.wav")


class Enemy(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = enemy_image
        self.rect = self.image.get_rect()
        self.reset()

    def reset(self):
        self.rect.midbottom = (random.randint(20, 780), random.randint(-500, -40))
        self.speed = random.randint(1, 2)

    def update(self):
        self.rect.y += self.speed
        if self.rect.top > 600:
            self.reset()


player = player_image.get_rect(midbottom=(400, 590))
speed = 6
bullets = []
cooldown = 0

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT]:
        player.x -= speed
    if keys[pygame.K_RIGHT]:
        player.x += speed
    player.clamp_ip(screen.get_rect())

    if cooldown > 0:
```

```
        cooldown -= 1
    if keys[pygame.K_SPACE] and cooldown == 0:
        bullets.append(bullet_image.get_rect(midbottom=player.midtop))
        shoot_sound.play()
        cooldown = 15

    for bullet in bullets:
        bullet.y -= 10
    bullets = [b for b in bullets if b.bottom > 0]

    screen.blit(background, (0, 0))
    for bullet in bullets:
        screen.blit(bullet_image, bullet)
    screen.blit(player_image, player)
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

Un Group maneja muchos sprites a la vez

Con el molde listo, fabrica ocho enemigos, antes del bucle. Esta línea es como la lista de balas que filtraste, pero sin corchetes: crea un Enemy ocho veces, y los mete todos en un Group. El guion bajo es una variable que no vas a usar: solo cuenta las vueltas. En el bucle, después de mover las balas, enemies.update() llama al update de cada enemigo del grupo. Una sola línea mueve a los ocho, cada uno a su velocidad. Y enemies.draw(screen) los dibuja a todos, con la imagen y el Rect de cada uno. Va justo después del fondo, porque si fuera antes, el fondo los taparía. Si mañana quieres veinte enemigos, solo cambias el ocho.

Tu `game.py` queda así — las líneas resaltadas son las nuevas o cambiadas (archivo `paso-3.py`):

```
import random

import pygame

pygame.init()
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("Most Wanted UFO Mini")
clock = pygame.time.Clock()

background = pygame.image.load("assets/background_4.png").convert()
player_image = pygame.image.load("assets/player_1.png").convert_alpha()
bullet_image = pygame.image.load("assets/Bullet.gif").convert_alpha()
enemy_image = pygame.image.load("assets/enemy_1.png").convert_alpha()
shoot_sound = pygame.mixer.Sound("assets/shoot.wav")


class Enemy(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = enemy_image
        self.rect = self.image.get_rect()
        self.reset()

    def reset(self):
        self.rect.midbottom = (random.randint(20, 780), random.randint(-500, -40))
        self.speed = random.randint(1, 2)
```

```
    def update(self):
        self.rect.y += self.speed
        if self.rect.top > 600:
            self.reset()


player = player_image.get_rect(midbottom=(400, 590))
speed = 6
bullets = []
cooldown = 0
enemies = pygame.sprite.Group(Enemy() for _ in range(8))

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT]:
        player.x -= speed
    if keys[pygame.K_RIGHT]:
        player.x += speed
    player.clamp_ip(screen.get_rect())

    if cooldown > 0:
        cooldown -= 1
    if keys[pygame.K_SPACE] and cooldown == 0:
        bullets.append(bullet_image.get_rect(midbottom=player.midtop))
        shoot_sound.play()
        cooldown = 15

    for bullet in bullets:
        bullet.y -= 10
    bullets = [b for b in bullets if b.bottom > 0]
    enemies.update()

    screen.blit(background, (0, 0))
    enemies.draw(screen)
    for bullet in bullets:
        screen.blit(bullet_image, bullet)
    screen.blit(player_image, player)
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

Los enemigos caen y vuelven a aparecer

Ejecuta el juego. Los enemigos bajan, cada uno a su velocidad, y cuando salen por abajo vuelven a aparecer arriba. En esta demo el ovni apunta y dispara solo, pero mira: las balas los atraviesan. Todavía nadie revisa si una bala choca con un enemigo; eso es la próxima lección.

Pruébalo:

```
python game.py
```

Lo que aprendiste

Repasemos. Una clase es un molde para crear objetos. __init__ prepara cada enemigo nuevo, y self es ese enemigo, con sus propios datos. Sprite y Group ya saben dibujar y actualizar a muchos a la vez. Y random hace que cada partida sea distinta. Tu ejercicio: agrega a Enemy un movimiento de lado, para que caigan en zigzag. En la próxima lección, las balas por fin aciertan.

- Una clase es un molde para crear objetos
- self es cada objeto con sus datos
- Sprite y Group ya saben dibujar y actualizar
- random da variedad

Resumen

- `class Enemy(pygame.sprite.Sprite):` define un molde; `Enemy()` crea un enemigo nuevo y ejecuta su `__init__`.
- `self` es el objeto concreto: `self.rect` y `self.speed` son de ese enemigo y de ningún otro.
- Un `Sprite` necesita `self.image` y `self.rect`; con eso `Group.draw` sabe dibujarlo.
- `pygame.sprite.Group(Enemy() for _ in range(8))` crea un grupo con 8 enemigos.
- `enemies.update()` llama al `update` de cada enemigo y `enemies.draw(screen)` los dibuja a todos.
- `random.randint(a, b)` da un entero al azar entre `a` y `b`, ambos incluidos.

Ejercicio

Empieza desde el `game.py` de esta lección.

- Haz que cada enemigo, además de caer, se mueva de lado a su propio ritmo y rebote al tocar los bordes.

Pista: cada enemigo necesita un dato nuevo en `self`, elegido en `reset`.

- Bonus: cambia el 8 por 15 y mira cómo cambia la dificultad.

Solución

```
def reset(self):
    self.rect.midbottom = (random.randint(20, 780), random.randint(-500, -40))
    self.speed = random.randint(1, 2)
    self.drift = random.choice([-1, 1])

def update(self):
    self.rect.y += self.speed
    self.rect.x += self.drift
    if self.rect.left < 0 or self.rect.right > 800:
        self.drift = -self.drift
    if self.rect.top > 600:
        self.reset()
```

Gráficos y sonidos: Most Wanted UFO Classic, de Jose Daniel Soto, licencia CC BY 4.0.